



Programovacie jazyky

Teória kategórií pre informatikov – ADT

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

11. apríla 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Úvod

Ako riešime veľké, zložité problémy?

- rozdeľ a panuj (divide et impera)
 1. rozdeľ na menšie, jednoduchšie, samostatné podproblémy
 - potrebujeme *rozložiteľnosť* (decomposability) problému
 2. vyrieš podproblémy
 - ideálne chceme *znovupoužiteľnosť* riešení aj v iných kontextoch
 3. zlož riešenie problému z riešení podproblémov
 - potrebujeme *zložiteľnosť* (composability) riešení podproblémov
- **abstrakcia** (zbavenie sa *zbytočných* detailov)
 - umožňuje stotožniť pôvodne rôzne kontexty
 - ale pozor na to, od čoho abstrahujeme

Teória kategórií – motivácia

- jedna z najabstraktnejších častí matematiky
- myšlienky z nej sa dajú implementovať v jazyku Haskell
 - môžeme ich „pretlačiť“ aj do iných, ešte menej abstraktných jazykov (praktická motivácia)
- abstrahuje od detailov $\Rightarrow$ zjednocuje veľa vecí ako z programátorského, tak aj z matematického hľadiska:
 - veci môžu byť totožné alebo môžu byť rôzne, ale pre dané účely je ich možné považovať za rovnaké (izomorfné)
 - homotopy type theory (sa pokúša riešiť tento jeden problém)
 - algebra, teória množín, topológia
 - Curry-Howard-Lambek izomorfizmus:
 - intuitionistic type theory (Martin-Löf type theory)
riešenie paradoxov v matematike (množina všetkých množín, „Holič holí práve tých, čo sa neholia samy.“ – Holí seba?)
 - výroková logika
 - cartesian closed categories

Čo dá teória kategórií informatikom?

- algebraické dátové typy
- funktory
- monády (monads)
- currying
- ...

Kategória

Čo tvorí kategóriu $\mathcal{C}$?

- *kolekcia* $\text{ob}(\mathcal{C})$ objektov: $A, B, C, \dots$
- *kolekcia* $\text{mor}(\mathcal{C})$ morfizmov: $f, g, h, \dots$ – aka šípky (arrows)
 - s každým morfizmom f sú spojené dva objekty (zdroj, cieľ) z $\mathcal{C}$
 - nazývajú sa $\text{dom}(f)$ – *doména* f a $\text{cod}(f)$ – *kodoména* f
 - ak $\text{dom}(f) = A$ a $\text{cod}(f) = B$, tak píšeme, $f : A \rightarrow B$
 - $\mathcal{C}(A, B)$ je *kolekcia* morfizmov z A do B v $\mathcal{C}$ (aka hom-set)
- operácia $\circ$ skladania morfizmov (h nazývame zložením f a g)
 - $\forall f : A \rightarrow B$ a $\forall g : B \rightarrow C$ existuje $h : A \rightarrow C$, že $h = g \circ f$
- $\forall A \in \text{ob}(\mathcal{C})$ existuje identický morfizmus $\text{id}_A : A \rightarrow A$

spĺňajúce nasledovné dva zákony:

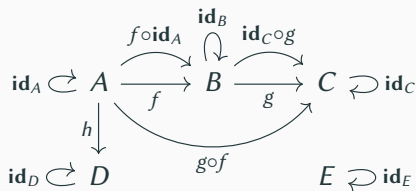
(neutrálny prvok – unit laws) $\forall f : A \rightarrow B$ platí:

$$f \circ \text{id}_A = f \text{ (right identity) a } \text{id}_B \circ f = f \text{ (left identity)}$$

(asociativita) $\forall f : A \rightarrow B, \forall g : B \rightarrow C$ a $\forall h : C \rightarrow D$ platí:

$$h \circ (g \circ f) = (h \circ g) \circ f$$

Príklad znázornenia kategórie



- kategória $\mathcal{C}$ je **malá** (small) $\Leftrightarrow \text{ob}(\mathcal{C})$ a $\text{mor}(\mathcal{C})$ sú množiny
- **veľká** (large) kategória $\Leftrightarrow$ nie je malá
- $\mathcal{C}$ je **lokálne malá** $\Leftrightarrow \forall A, B \in \text{ob}(\mathcal{C}) : \mathcal{C}(A, B)$ je množina
 - malá kategória $\Rightarrow$ lokálne malá kategória

Základné malé kategórie

- prázdna kategória 0 : bez objektov, čiže aj bez morfizmov
- kategória 1 :
 - $\text{id}_A \hookrightarrow A$
- kategória 2 :
 - $\text{id}_A \hookrightarrow A \xrightarrow{f} B \rightrightarrows \text{id}_B$
- monoid $M = (S = \{e, x_2, \dots, x_n\}, \otimes : S \times S \rightarrow S)$
 - e je neutrálny prvok (vzhľadom na $\otimes$) a $\otimes$ je asociatívne
 - $\text{ob}(\mathcal{M}) = \{m\}$, $\text{mor}(\mathcal{M}) = \mathcal{M}(m, m) = S$, $x_i \circ x_j = x_i \otimes x_j$

$$\text{id}_m = e \hookrightarrow m \rightrightarrows x_n$$

$\uparrow$
 x_2

.....

- Hovoríme, že kategória $\mathcal{M}$ je asociovaná s monoidom M .
- Zoznamy s operáciou spojenia tvoria monoid (nekomutatívny):

```
1 instance Monoid [a] where
2 mempty  = []
3 mappend = (++)
```

Usporiadania ako malé kategórie

- predusporiadanie (preorder) $P = (S = \{x_1, \dots, x_n\}, \leq)$
 - $\leq$ je reflexívna a tranzitívna relácia (podmnožina $S \times S$)
 - relácie ekvivalencie $\Rightarrow$ predusporiadanie, ale $\nRightarrow$ čiastočné usp.
 - $\text{ob}(\mathcal{P}) = S$
 - $\mathcal{P}(x_i, x_j) = \begin{cases} \{(x_i; x_j)\} & \text{ak } x_i \leq x_j \\ \emptyset & \text{ak } x_i \not\leq x_j \end{cases}$
 - $\text{id}_{x_i} = (x_i; x_i)$ a $(x_j; x_k) \circ (x_i; x_j) = (x_i; x_k)$
 - Hovoríme, že kategória $\mathcal{P}$ je asociovaná s predusporiadaním P .

Poznámka

Kategóriu $\mathcal{P}$ je rovnakým spôsobom možné asociovať aj k čiastočnému (predusporiadanie + antisymetria) alebo úplnému usporiadaniu P .

Príklad: relácia byť podmnožinou tvorí čiastočné usporiadanie.

Definícia

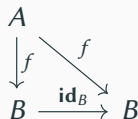
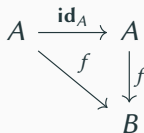
Symbolom $[n]$, pre $n \in \mathbb{N}_0$, označujeme kategóriu asociovanú s obvyklým úplným usporiadaním množiny $\{0, 1, \dots, n\}$.

Komutatívny diagram – neutrálnosť identických morfizmov

$$\text{id}_A \hookrightarrow A \xrightarrow{f} B \hookleftarrow \text{id}_B$$

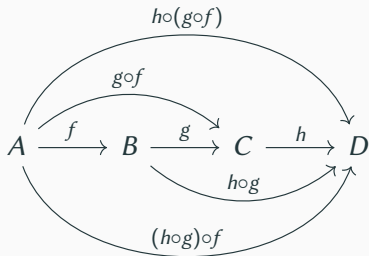
$$f \circ \text{id}_A = f \quad \text{a} \quad \text{id}_B \circ f = f$$

Komutatívny diagram



- Ak niektoré identické morfizmy v kategórii
 - chýbajú alebo
 - nespĺňajú podmienku neutrálnosti,
- tak sa nazáva polokategória.
 - semicategory
 - non-unital category

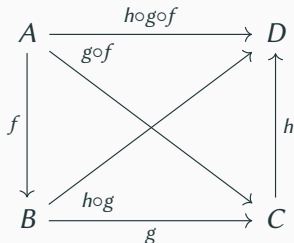
Komutatívny diagram – asociativita ○



$$h \circ (g \circ f) = (h \circ g) \circ f$$

- Asociativita zjednodušuje zápis a prácu s operáciou skladania.
 - nie je to nutné, uvažuje sa aj s oslabením asociativity
 - namiesto identity ($=$) stačí izomorfizmus ($\equiv$)

Komutatívny diagram



- objekty sú množiny
 - *abstrahujeme* od toho aké prvky ju tvoria
- morfizmy sú totálne (úplné) funkcie medzi množinami
 - nemusia byť surjektívne, ani injektívne
 - *zabudneme* aké relácie sa ukrývajú za morfizmami
- operácia $\circ$ je skladanie funkcií
 - *zapamätáme* si len ako je definované $\circ$ na morfizmoch
- aj tak vieme zistiť
 - ktoré morfizmy predstavujú
 - injektívne funkcie
 - surjektívne funkcie
 - bijektívne funkcie
 - ktoré objekty predstavujú
 - prázdne množiny
 - jedno-prvkové množiny
- ale potrebujeme uvažovanie na vyššej úrovni abstrakcie

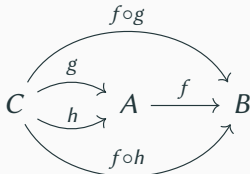
Mono-, epi- a izo-morfizmus

Monomorfizmus

Definícia

Morfizmus $f : A \rightarrow B$ nazývame **monomorfizmus** práve vtedy, ak pre všetky objekty C a všetky morfizmy $g : C \rightarrow A$ a $h : C \rightarrow A$ platí:

$$f \circ g = f \circ h \implies g = h.$$



Poznámka

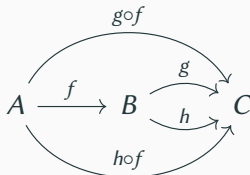
V kategórii *Sets* platí, že f je monomorfizmus $\Leftrightarrow f$ je injekcia.

Epimorfizmus

Definícia

Morfizmus $f : A \rightarrow B$ nazývame **epimorfizmus** práve vtedy, ak pre všetky objekty C a všetky morfizmy $g : B \rightarrow C$ a $h : B \rightarrow C$ platí:

$$g \circ f = h \circ f \implies g = h.$$



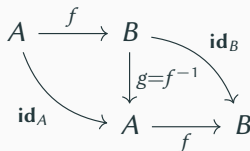
Poznámka

V kategórii *Sets* platí, že f je epimorfizmus $\Leftrightarrow f$ je surjekcia.

Izomorfizmus

Definícia

Morfizmus $f : A \rightarrow B$ nazývame **izomorfizmus** práve vtedy, ak existuje morfizmus $g : B \rightarrow A$, taký, že: $g \circ f = \text{id}_A$ a $f \circ g = \text{id}_B$.



Poznámka

- Pretože g pre f je jednoznačne určené, tak g označujeme aj f^{-1} .
- Každý identický morfizmus je izomorfizmus.
- V kategórii *Sets* platí: f je izomorfizmus $\Leftrightarrow f$ je bijekcia.

monomorfizmus $\wedge$ epimorfizmus $\nRightarrow$ izomorfizmus

Definícia

Kategória $\mathcal{C}$ je **tenká** (thin) $\Leftrightarrow \forall A, B \in \text{ob}(\mathcal{C}) : |\mathcal{C}(A, B)| \leq 1$.

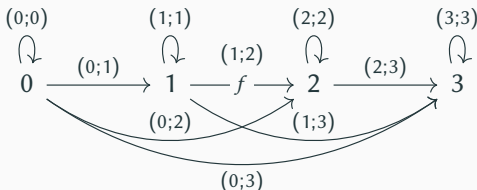
Kategória, ktorá nie je tenká je **hrubá** (thick).

Poznámka

Kategória asociovaná s predusporiadaním je tenká kategória.

Príklad

V kategórii $[3]$ je f monomorfizmus aj epimorfizmus, ale f *nie je* izomorfizmus.



Typy $\leftrightarrow$ množiny (Haskell)

- **Void** $\equiv$ prázdna množina $\emptyset \equiv 0$
 - $\text{id}_{\text{Void}} : \text{Void} \rightarrow \text{Void}$
 - potrebujeme aby existovala (aj keď sa nedá zavolať)
 - **absurd** $:: \text{Void} \rightarrow a$
 - **Data.Void** – „A Haskell 98 logically uninhabited data type“
 - funkcia $f : \text{Int} \rightarrow \text{Void}$ neexistuje
- Unit (označuje sa „()“) $\equiv$ jednoprvková množina $\{e_1\} \equiv 1$
 - $\{f :: a \rightarrow (); f\ x = ()\}$
 - **f** $:: () \rightarrow \text{Int}$; napríklad: **f** **()** = 42
 - prvky množiny A možno nahradiť funkciami $f_a : () \rightarrow A; f_a() = a$
 - f_a sa nazýva zovšeobecnený (generalized) prvok objektu A
- **Bool** $\equiv$ dvojprvková množina $\{e_T, e_F\}$
 - **f** $:: \text{Bool} \rightarrow \text{Int}$, **f** $:: \text{Int} \rightarrow \text{Bool}$
 - funkcie vracajúce **Bool** sa nazývajú predikáty

Poznámka (pre Haskell)

Každý typ má aj dolníka (bottom) „ $\perp$ “. Tento detail zanedbáme.

Terminálny a inícálny objekt

Úvahy o objektoch kategórie *Sets*

Ako definujeme objekty, ktoré predstavujú:

- prázdne množiny
- jedno-prvkové množiny
- karteziánsky súčin dvoch množín

keď nemôžeme:

- hovoriť o elementoch množín, ktoré sa skrývajú za objektami
 - t.j. nemôžeme sa pozeráť do objektov

Poznámka

Musíme použiť iba vlastnosti objektov vzhľadom na iné objekty (t.j. vzťahy medzi objektami definované morfizmami, čiže šípkami).

Univerzálna konštrukcia

- definujeme vzor (pomocou šípok a objektov)
- vyhľadáme ho v kategórii
- nájdeme zhody, môže ich byť:
 - 0 – hľadaný objekt neexistuje
 - 1 – našli sme ho
 - viac, až nekonečne veľa – musíme vybrať najlepší
- preto definujeme čiastočné usporiadanie na zhodách
 - $\exists!$ šípka (vybraným smerom); v zmysle tohto usporiadania
 - hľadáme *najväčšiu* (najlepšiu) zhodu z , t.j. $\forall z' : z' \leq z$
 - nie maximálnu hodnotu (t.j. nikto nie je lepší), lebo
 - môže byť viac neporovnateľných lokálnych maxím
- najväčší prvok nemusí existovať
 - potom objekt ktorý hľadáme tiež neexistuje
- pokiaľ existuje zhoda s najväčšou hodnotou
 - tak tento vzor definuje objekt ktorý hľadáme
 - môže ich byť viac, ale všetky sú rovnaké (až na izomorfizmus)

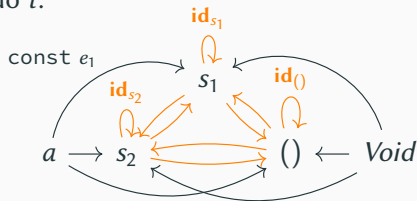
Terminálny objekt

Definícia

Objekt $t \in \text{ob}(\mathcal{C})$ nazývame **terminálny objekt** $\iff \forall x \in \text{ob}(\mathcal{C}) :$
existuje práve jeden morfizmus z x do t .

Vzor: **t** (objekt)

Usporiadanie: $z' \leq z \iff z' \xrightarrow{!} z$



Poznámka

- *terminálny objekt sa zvykne označovať 1 , $*$ alebo pt*
 - *jednotka vzhľadom na násobenie (karteziánsky súčin)*
 - *v Sets: terminálny objekt $\iff$ jedno-prvková množina*
- *ak existuje, tak je jediný až na izomorfizmus (jednoznač. určený)*
- *morfizmus $f : 1 \rightarrow a$ sa nazýva **zovšeobecnený prvok** a*
- *Príklad: v kategórii $[3]$ je terminálny objekt 3 .*

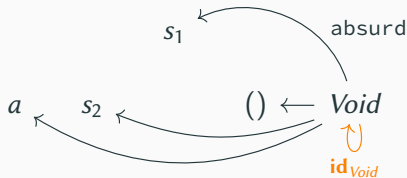
Iniciálny objekt

Definícia

Objekt $i \in \text{ob}(\mathcal{C})$ nazývame **iniciálny objekt** $\iff \forall x \in \text{ob}(\mathcal{C}) :$
existuje práve jeden morfizmus z i do x .

Vzor: ***i*** (objekt)

Usporiadanie: $z' \leq z \iff z' \xleftarrow{!} z$



Poznámka

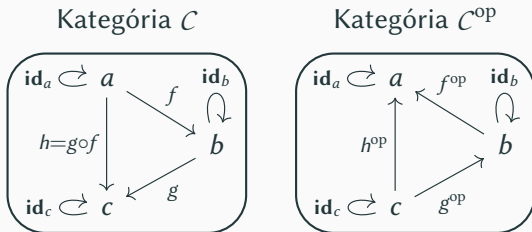
- *iniciálny objekt sa zvykne označovať 0*
 - *nula vzhľadom na násobenie (karteziánsky súčin)*
 - *v Sets: terminálny objekt $\iff$ prázdna množina*
- *ak existuje, tak je jediný až na izomorfizmus (jednoznač. určený)*
- *Príklad: v kategórii $[3]$ je iniciálny objekt 0.*

Opačná kategória (opposite category)

Definícia

Hovoríme, že kategória $\mathcal{C}^{\text{op}}$ je **opačná kategória** ku kategórii $\mathcal{C}$ práve vtedy, keď vznikne z $\mathcal{C}$ nasledovnou konštrukciou:

- zachováme objekty, otočíme smer šípiek ($f \rightsquigarrow f^{\text{op}}$)
- obrátime poradie skladania morfizmov: $f^{\text{op}} \circ g^{\text{op}} = (g \circ f)^{\text{op}}$



Poznámka

$(\mathcal{C}^{\text{op}})^{\text{op}} = \mathcal{C}$. Ak $\mathcal{C} = \mathcal{C}^{\text{op}}$, tak hovoríme, že $\mathcal{C}$ je **samoduálna**.

Definícia (koštruktúra)

Hovoríme, že x je koštruktúra v $\mathcal{C} \iff x$ je štruktúra v $\mathcal{C}^{\text{op}}$.

Príklad

- Koterminálny prvok v $\mathcal{C}$ je terminálny prvok v $\mathcal{C}^{\text{op}}$.
- Iniciálny prvok možno definovať ako koterminálny.

Definícia (duálne pojmy)

Štruktúra x v $\mathcal{C}$ je duálna ku štruktúre y v $\mathcal{C} \iff kox = y$.

Príklad (duálne pojmy)

- monomorfizmus $\bowtie$ epimorfizmus
- terminálny objekt $\bowtie$ iniciálny objekt

Definícia (dualita)

V kategórii $\mathcal{C}$ platí tvrdenie $\alpha \iff$ v $\mathcal{C}^{\text{op}}$ platí tvrdenie α^{op} .

Súčin a súčet

Súčin (product)

Definícia

Súčinom objektov a a b z kategórie $\mathcal{C}$ je objekt c spolu s dvojicou morfizmov $\pi_a : c \rightarrow a$ a $\pi_b : c \rightarrow b$, pre ktoré platí:

$$\forall c', \pi'_a : c' \rightarrow a, \pi'_b : c' \rightarrow b; \exists ! f : c' \rightarrow c; \pi_a \circ f = \pi'_a \wedge \pi_b \circ f = \pi'_b.$$

Vzor:

$$\begin{array}{ccc} & \textcolor{brown}{c} & \\ \pi_a = \text{fst} \swarrow & & \searrow \text{snd} = \pi_b \\ a & & b \end{array}$$

Usporiadanie: $u \leq v \Leftrightarrow u \overset{!}{\dashrightarrow} v$

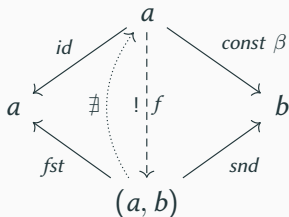
$$\begin{array}{ccccc} & & c' & & \\ & \swarrow \pi'_a & \vdots f & \searrow \pi'_b & \\ a & \xleftarrow{\pi_a} & c & \xrightarrow{\pi_b} & b \end{array}$$

Poznámka

- Objekt c spolu s morfizmami π_a a π_b označujeme $a \times b$.
 - Morfizmy π_a a π_b nazývame kanonické projekcie.
- Súčin dvoch objektov je, až na izomorfizmus, jednoznačne určený.
- V kategórii *Sets* predstavuje operácia $\times$ karteziánsky súčin.
- Súčin nemusí existovať pre každé dva objekty (alebo vôbec).

Súčin (product) – príklad v $\mathcal{Hask}$ pre nesurjektívne f

Trojica $(a, id, const \beta)$ je horší produkt než $((a, b), fst, snd)$:



$$fst \circ f = id$$

$$snd \circ f = const \beta$$

$$fst \ (x, _) = x$$

$$snd \ (_, y) = y$$

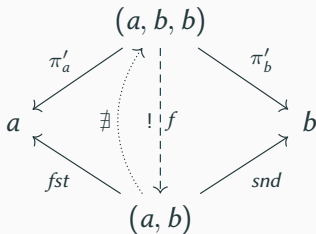
$$f \ x = (x, \beta)$$

Poznámka

Dá sa povedať, že a je horšie než (a, b) lebo zahadzuje informáciu o b .

Súčin (product) – príklad v $\mathcal{Hask}$ pre neinjektívne f

Trojica $((a, b, b), \pi_a, \pi_b)$ je horší produkt než $((a, b), fst, snd)$:



$$fst \circ f = \pi'_a$$

$$snd \circ f = \pi'_b$$

$$fst \ (x, _) = x \qquad \pi'_a \ (x, _, _) = x$$

$$snd \ (_, y) = y \qquad \pi'_b \ (_, _, y) = y$$

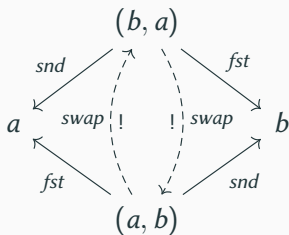
$$f \ (x, _, y) = (x, y)$$

Poznámka

Dá sa povedať, že (a, b, b) je horšie než (a, b) lebo obsahuje redundantnú (zbytočnú) informáciu.

Súčin (product) – príklad v $\mathcal{Hask}$ pre bijektívne f

$((b, a), snd, fst)$ je rovnako dobrý produkt ako $((a, b), fst, snd)$:



$$fst \circ swap = snd$$

$$snd \circ swap = fst$$

$$fst \ (x, _) = x \qquad snd \ (_, y) = y$$

$$swap \ (y, x) = (x, y)$$

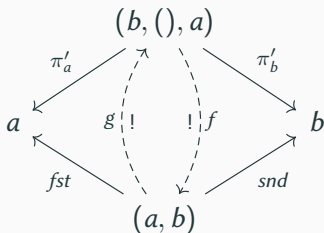
Poznámka

Dá sa povedať, že (b, a) je rovnako dobrý produkt ako (a, b) , lebo obsahuje rovnakú informáciu, len inak zapísanú.

Funkcia $swap$ je izomorfizmus medzi týmito reprezentáciami.

Súčin (product) – iný príklad v $\mathcal{Hask}$ pre bijektívne f

$((b, (), a), \pi_a, \pi_b)$ je rovnako dobrý produkt ako $((a, b), fst, snd)$:



$$fst \circ f = \pi'_a$$

$$snd \circ f = \pi'_b$$

$$\pi_a \circ g = fst$$

$$\pi_b \circ g = snd$$

$$fst \ (x, _) = x$$

$$\pi'_a \ (_, _, x) = x$$

$$snd \ (_, y) = y$$

$$\pi'_b \ (y, _, _) = y$$

$$f \ (y, _, x) = (x, y)$$

$$g \ (x, y) = (y, (), x)$$

Poznámka

Dá sa povedať, že $(b, (), a)$ je rovnako dobrý produkt ako (a, b) , lebo obsahuje rovnakú informáciu, len inak zapísanú.

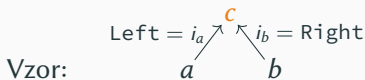
Funkcia f (aj g) je izomorfizmus medzi týmito reprezentáciami.

Súčet (sum)

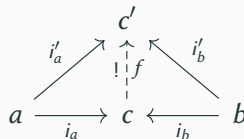
Definícia

Súčtom objektov a a b z kategórie $\mathcal{C}$ je objekt c spolu s dvojicou morfizmov $i_a : a \rightarrow c$ a $i_b : b \rightarrow c$, pre ktoré platí:

$$\forall c', i'_a : a \rightarrow c', i'_b : b \rightarrow c'; \exists ! f : c \rightarrow c'; f \circ i_a = i'_a \wedge f \circ i_b = i'_b.$$



Usporiadanie: $u \leq v \Leftrightarrow u \xleftarrow{!} v$



Poznámka

- Objekt c spolu s morfizmami i_a a i_b označujeme $a \sqcup b$.
 - i_a a i_b nazývame kanonické injekcie (nesúvisí s monomorfizmom)
- Súčet dvoch objektov je, až na izomorfizmus, jednoznačne určený.
- Súčet v *Sets* je disjunktné zjednotenie (disjoint/tagged union).
- Kategorický súčet je kategorický kosúčin (coproduct).

Algebraické datové typy

- Aká algebra?
- Hovoríme predsa o typoch.
- Prečo teda algebraické dátové typy?
- Existuje niečo ako algebra typov?

ÁNO!

Algebraické dátové typy – súčin (komutatívnosť, asociatívnosť)

Správa sa súčin $a \times b = (a, b)$ v kategórii $\mathcal{Hask}$ ako násobenie?

- komutatívnosť: $(a, b) \equiv (b, a)$
 - $(a, b) \neq (b, a)$ (sú rôzne typy)
 - ale sú izomorfné (`swap` je izomorfizmus)

```
1 swap :: (a, b) -> (b, a)
2 swap pair = (snd pair, fst pair)
```

- asociatívnosť: $((a, b), c) \equiv (a, (b, c))$
 - $((a, b), c) \neq (a, (b, c))$
 - tiež sú izomorfné (`assoc` je izomorfizmus)

```
1 assoc :: ((a, b), c) -> (a, (b, c))
2 assoc ((x, y), z) = (x, (y, z))
```

Algebraické dátové typy – súčin (neutrálny prvok)

Správa sa súčin $a \times b = (a, b)$ v kategórii $\mathcal{Hask}$ ako násobenie?

- neutrálny prvok $e_x = 1$: $(a, ()) \equiv a \equiv ((), a)$
 - $(a, ()) \neq a, ((), a) \neq a$
 - ale sú izomorfné (`runit` a `lunit` sú izomorfizmy)

```
1  runit :: (a, ()) -> a
2  runit pair = fst pair
3
4  irunit :: a -> (a, ())
5  irunit x = (x, ())
6
7  lunit :: ((), a) -> a
8  lunit pair = snd pair
9
10 ilunit :: a -> ((), a)
11 ilunit x = ((), x)
```

Algebraické dátové typy – súčin (komutatívny monoid)

⇒ Súčin v $\mathcal{Hask}$ tvorí komutatívny monoid (až na izomorfizmus)

- predstavuje n -tice (tuples), záznamy (records)
 - vďaka asociativite nemusíme písať vnútorné zátvorky
 - $((a, b), c) = (a, (b, c)) = (a, b, c)$
 - v praxi operátor $(,)$ nie je komutatívny ani asociatívny, ale n -tice nemusíme písať pomocou dvojíc

Algebra	Kategória $\mathcal{Hask}$	Haskell
$a * 1 = a$	$a \times 1 \equiv a$	$(a, ()) \equiv a$
$1 * a = a$	$1 \times a \equiv a$	$((), a) \equiv a$
$(a * b) * c =$ $= a * (b * c)$	$(a \times b) \times c \equiv$ $\equiv a \times (b \times c)$	$((a, b), c) \equiv$ $\equiv (a, (b, c))$
$a * b = b * a$	$a \times b \equiv b \times a$	$(a, b) \equiv (b, a)$

Algebraické dátové typy – súčet

⇒ Súčet v $\mathcal{Hask}$ tvorí komutatívny monoid (až na izomorfizmus)

- predstavuje tagged union, variantné záznamy (variant records)
 - $a + b = \text{Either } a \ b$
 - **data Either** $a \ b = \text{Left } a \mid \text{Right } b$
 - enumerovaný typ je špeciálny prípad tagged union:
 - **data Color** $= \text{Red} \mid \text{Green} \mid \text{Blue}$
 - **data Color** $= \text{Red } () \mid \text{Green } () \mid \text{Blue } ()$

Algebra	Kategória $\mathcal{Hask}$	Haskell
$a + 0 = a$	$a + 0 \equiv a$	Either $a \ \text{Void} \equiv a$
$0 + a = a$	$0 + a \equiv a$	Either Void $a \equiv a$
$(a + b) + c =$ $= a + (b + c)$	$(a + b) + c \equiv$ $\equiv a + (b + c)$	Either $(\text{Either } a \ b) \ c \equiv$ $\equiv \text{Either } a \ (\text{Either } b \ c)$
$a + b =$ $= b + a$	$a + b \equiv$ $\equiv b + a$	Either $a \ b \equiv$ $\equiv \text{Either } b \ a$

Algebraické dátové typy – polookruh (semiring, rig)

nemáme inverznú operáciu k $+$ a $*$, ale násobenie je distributívne
a $e_+ = 0$ je absorbujúci prvok pre operáciu $*$ (oboje zľava aj sprava)

Algebra	Kategória $\mathcal{Hask}$	Haskell
$a * 0 = 0$	$a \times 0 \equiv 0$	$(a, \text{Void}) \equiv \text{Void}$
$a * (b + c) =$ $= a * b + a * c$	$a \times (b + c) \equiv$ $\equiv a \times b + a \times c$	$(a, \text{Either } b \ c) \equiv$ $\equiv \text{Either } (a, b) \ (a, c)$

- 1 `distr :: (a, Either b c) -> Either (a, b) (a, c)`
- 2 `distr (x, Left y) = Left (x, y)`
- 3 `distr (x, Right y) = Right (x, y)`

Príklady algebraických vzťahov

Algebra	Haskell
$1 + 1 = 2$	<code>Either () () $\equiv$ Bool</code>
$1 + a$	<code>Either () a</code> <code>data Maybe a = Nothing Just a</code>

Algebraické rovnice – zoznam

Algebra	Haskell
$L(a) = 1 + a * L(a)$	<pre>data L a = Either () (a, L a) data [] a = [] a : [a]</pre>
$L(a) - a * L(a) = 1$ $L(a) * (1 - a) = 1$ $L(a) = \frac{1}{1-a} = \sum_{n=0}^{\infty} a^n =$ $= 1 + a + a * a + a * a * a + \dots$	odčítateľ a deliť v polookruhu nevieme, ale vieme dosadzovať a nájsť pevný bod
$L(a) = 1 + a * (1 + a * L(a))$ $= 1 + a + a * a * (1 + a * L(a))$ $= 1 + a + a * a + a * a * a + \dots$	<pre>data [] a = L0 L1 a L2 a a L3 a a a ...</pre>

Algebraické rovnice – strom

Algebra	Haskell
$T(a) = 1 + a * T(a)^2$	<pre>data T a = Either () (a, T a, T a) data T a = Empty Node a (T a) (T a)</pre>
$a * T(a)^2 - T(a) + 1 = 0$ $T(a) = \frac{1 - \sqrt{1 - 4a}}{2a}$ Taylorov rozvoj $T(a)$ pre $a = 0$: $T(a) = 1 + a + 2a^2 + 5a^3 + 14a^4 + \dots$	odčítateľ, deliť a odmocňovať v polookruhu nevieme, ale vieme dosadzovať a nájsť pevný bod
$T(a) = \sum_{k=0}^{\infty} \binom{2k}{k} \frac{a^k}{k+1}$ (pozri: Catalanove čísla)	<pre>data T a = T0 T1 a T2 Bool a a ...</pre>